

گزارش کار پروژه

پیاده‌سازی دیجیتال بازی رومیزی Unmatched

یک موتور بازی کارتی داده‌محور با C++17

کیارش حبیبی
یوسف پاک

۱. مقدمه

این گزارش، مستندسازی کامل پروژه‌ی «Unmatched Card Game Engine» است؛ پیاده‌سازی نرم‌افزاری بازی رومیزی محبوب Unmatched (با دو شخصیت Dracula و Sherlock Holmes) با زبان C++ و کتابخانه‌ی گرافیکی raylib. هدف از این پروژه، طراحی و پیاده‌سازی یک موتور بازی مازولار و داده‌محور بود؛ به‌گونه‌ای که قوانین، شخصیت‌ها، کارت‌ها و نقشه‌های بازی مستقیماً در کد ++C هاردکد نشوند، بلکه از فایل‌های JSON خوانده شوند.

در ادامه‌ی این گزارش، ابتدا اهداف پروژه، سپس معماری کلی و لایه‌بندی آن، الگوهای طراحی به‌کاررفته، مکانیزم مدیریت خطا، نحوه‌ی build و اجرای پروژه، و در نهایت ساختار پوشه‌ها و توضیح فایل‌به‌فایل آن ارائه می‌شود.

۲. اهداف پروژه

- پیاده‌سازی کامل قوانین بازی Unmatched شامل حرکت، نبرد، اثرات کارت‌ها و شرایط پیروزی.
- طراحی معماری‌ای که منطق بازی (Model/Engine) کاملاً از رابط کاربری (View) جدا باشد.
- داده‌محور کردن محتوای بازی؛ اضافه‌کردن قهرمان یا کارت جدید فقط با افزودن فایل JSON، بدون تغییر کد ++C.
- طراحی یک سلسله‌مراتب استثنا (Exception) برای مدیریت خطاهای فایل، JSON و داده به‌شکلی ایمن و قابل‌ردیابی.
- استفاده از الگوهای طراحی استاندارد (Factory، Observer، Singleton، Strategy) برای افزایش توسعه‌پذیری کد.

۳. معماری کلی پروژه

کد پروژه به پنج لایه‌ی مجزا تقسیم شده که هرکدام مسئولیت مشخصی دارند. این جداسازی باعث می‌شود تغییر در یک لایه (مثلاً نحوه‌ی رسم صفحه) تأثیری بر منطق قوانین بازی نداشته باشد.

مسئولیت	پوشه	لایه
کلاس‌های داده‌ای ساده: Card، Fighter/Hero/Sidekick، Deck، Map، Tile. این لایه فقط وضعیت و دسترسی‌های ساده دارد و هیچ قانونی از بازی در آن پیاده نشده.	/model	Model
قطعات واقعی قوانین بازی: Effect (کاری که یک کارت انجام می‌دهد)، Condition (شرط اعمال شدن یک اثر)، Query (مقداری محاسبه شده که یک اثر می‌خواند) و gameData (زمینه‌ای که بین این سه رد و بدل می‌شود).	/engine	Engine
تبدیل فایل‌های JSON زیر data/ به اشیای واقعی Hero/Card/Map/Effect. تنها لایه‌ای که فرمت داده روی دیسک را می‌شناسد.	/factory	Factory
GameManager (موتور قوانین / ماشین حالت نوبت‌ها)، SceneManager (صحنه‌ی فعال)، PlayerSelectionManager (انتخاب قهرمان پیش از شروع بازی)، AudioManager/FontManager/TextureManager (کش منابع)، و app (حلقه‌ی اصلی raylib).	/controller	Controller
صحنه‌های رسم شده با raylib (MainScene، HeroSelectionScene، GameScene) و اجزای رابط کاربری کوچک (Button، Particle). صحنه‌ها فقط وضعیت GameManager را می‌خوانند و رسم می‌کنند؛ هیچ قانونی در آن‌ها نیست.	/view	View

جریان داده برای قوانین بازی همواره یک‌طرفه است: فایل Factory → JSON → شیء Engine → Model → GameManager (Effect/Condition/Query) → اعمال نتیجه توسط → خواندن وضعیت و رسم آن توسط View.

۴. الگوهای طراحی استفاده شده

۴.۱ Factory

به ازای هر نوع داده، یک کلاس Factory مجزا وجود دارد: HeroFactory، CardFactory، DeckFactory، MapFactory، TileFactory، SidekickFactory، EffectFactory، ConditionFactory، QueryFactory و HeroInfoFactory. با این طراحی، افزودن یک قهرمان یا کارت جدید تنها به افزودن یک فایل JSON زیر data/ نیاز دارد، بدون هیچ تغییری در کد ++C.

Observer ۴.۲

رابط IGameObserver به GameScene اجازه می‌دهد به رویدادهایی مانند onCardPlayed، onCombatResolved و onFighterDamaged که توسط GameManager فراخوانی می‌شوند واکنش نشان دهد؛ بدون این‌که GameManager هیچ اطلاعی از لایه‌ی رسم داشته باشد.

۴.۳ چندریختی به سبک Strategy

کلاس‌های Effect، Condition و Query هرکدام یک کلاس پایه‌ی انتزاعی هستند که پیاده‌سازی‌های مشخص زیادی دارند و در زمان بارگذاری، بر اساس فیلد type در JSON، توسط Factory مربوطه ساخته می‌شوند.

۴.۴ Singleton (به سبک Meyer's)

PlayerSelectionManager تنها Singleton عمدی پروژه است که برای انتقال اطلاعات «کدام بازیکن کدام قهرمان را انتخاب کرده» از صفحه‌ی انتخاب قهرمان به داخل بازی استفاده می‌شود. این Singleton به صورت یک static محلی داخل یک متد کلاس پیاده شده، نه یک متغیر سراسری (global) عمومی.

۴.۵ تزریق وابستگی (Dependency Injection)

کلاس app مالک AudioManager، FontManager، TextureManager و SceneManager به صورت unique_ptr است و اشاره‌گر خام آن‌ها را به هرکس که نیاز داشته باشد پاس می‌دهد؛ بدون این‌که هیچ‌کدام از این‌ها متغیر سراسری باشند.

۵. مدیریت استثناءها (Exception Handling)

به جای این که خطاها بی صدا نادیده گرفته شوند یا برنامه با `std::terminate()` و بدون پیام مفید متوقف شود، یک سلسله مراتب استثنای اختصاصی در `include/utility/exceptions.h` تعریف شده است:

AppException (پایه)

- امکان باز کردن یا لیست کردن فایل/پوشه وجود ندارد -- FileNotFoundException
- معتبر نیست JSON فایل باز شد اما -- JsonParseException
- پارس شد اما فیلد لازم وجود ندارد یا نوع آن اشتباه است JSON -- DataFormatException
- در ساخت شیء ناموفق بود؛ یکی از سه استثنای بالا Factory یک -- FactoryException

را با اطلاعات «چه چیزی در حال ساخته شدن بود» بسته بندی می کند

هر لایه هنگام عبور استثناً به لایه‌ی بالاتر، اطلاعات بیشتری به آن اضافه می‌کند و لایه‌ی صحنه (Scene) جایی است که برنامه واقعاً بازیابی می‌شود، نه فقط لاگ می‌کند:

- `utility/file.cpp` (توابع `load()` و `listFiles()`) در پایین‌ترین سطح، استثنای `FileNotFoundException` یا `JsonParseException` پرتاب می‌کند.
- هر `Factory::create()` خطاهای داخلی خودش را می‌گیرد و یک `FactoryException` با نام‌گذاری دقیق (مثلاً «ساخت قهرمان Dracula ناموفق بود: فیلد name در stats.json وجود ندارد») دوباره پرتاب می‌کند.
- `GameManager::createFromSelection()` اطلاعات «نقشه‌ی X در حال آماده‌سازی بود» را به هر خطای `Factory` اضافه می‌کند.
- هر `SceneManager::changeScene()` ناشی از ساخت یک صحنه را می‌گیرد، آن را لاگ می‌کند و به جای کرش کل برنامه، به منوی اصلی برمی‌گردد.
- `main()` آخرین لایه‌ی محافظتی است که هر خطای رسیدگی نشده را می‌گیرد و با پیامی واضح خارج می‌شود.

۶. نحوه‌ی Build و اجرای پروژه

پیش‌نیازها: CMake نسخه‌ی ۳.۱۶ به بالا، یک کامپایلر با پشتیبانی از C17++ و (در لینوکس) هدرهای توسعه‌ی X11/OpenGL که raylib به آن‌ها نیاز دارد (libx11-dev، libxrandr-dev، libxinerama-dev، libxcursor-dev، libxi-dev، libgl-dev). کتابخانه‌های raylib و nlohmann/json به‌صورت خودکار توسط CMake از طریق FetchContent دریافت می‌شوند و نیازی به نصب دستی آن‌ها نیست.

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
cmake --build . -j$(nproc)
./CardGameEngine
```

در زمان build، پوشه‌های /assets و /data به‌طور خودکار کنار فایل اجرایی کپی می‌شوند (طبق تنظیمات CMakeLists.txt).

۷. ساختار پوشه‌ها و توضیح فایل‌به‌فایل

در این بخش، محتوای هر پوشه‌ی اصلی پروژه و نقش فایل‌های مهم آن توضیح داده می‌شود.

۷.۱ src/controller و include/controller

فایل	توضیح
app.h	کلاس اصلی برنامه؛ مالک SceneManager و مدیرهای منابع، اجراکننده‌ی حلقه‌ی اصلی raylib.
GameManager.h/.cpp	موتور قوانین بازی؛ ماشین‌حالت نوبت‌ها، اعمال کارت‌ها و اثرات، تشخیص برنده.
SceneManager.h	مدیریت صحنه‌ی فعال و جابه‌جایی امن بین صحنه‌ها با پشتیبانی از بازیابی خطا.
PlayerSelectionManager.h	Singleton نگه‌دارنده‌ی انتخاب قهرمان هر بازیکن پیش از شروع بازی.
AudioManager / FontManager / TextureManager	کش منابع صوتی، فونت و بافت که فقط یک‌بار بارگذاری می‌شوند.

۷.۲ src/model و include/model

این پوشه کلاس‌های داده‌ای خالص بازی را در بر دارد: Card، Fighter/Hero/Sidekick، Deck، Map و Tile، به‌همراه enum‌های کمکی زیر:

فایل	توضیح
heroList.h	enum class HeroList شامل Dracula و SherlockHolms؛ نام‌ها دقیقاً با نام پوشه‌های data / مطابقت دارند.
typeOfAttack.h	نوع حمله: melee یا ranged.
typeOfCard.h	نوع کارت: def، multipurpose، attack یا event.
typeOfEvent.h	زمان فعال‌شدن اثر یک کارت (before_combat، during_combat، after_combat و ...).
typeOfFighter.h / typeOfPerformer.h	برای فیلتر کردن جنگجویان و مشخص‌کردن این‌که چه کسی مجاز به بازی‌کردن یک کارت است.
InfoHero.h	داده‌ی نمایشی صرف برای صفحه‌ی انتخاب قهرمان، مجزا از کلاس Hero اصلی بازی.

include/engine ۷.۳

قطعات سازنده‌ی قوانین بازی؛ همه سرآمد (header-only) هستند و کلاس‌های نهایی توسط EffectFactory، ConditionFactory و QueryFactory از روی JSON ساخته می‌شوند.

توضیح	فایل
ساختار زمینه‌ی مشترکی که به هر Effect/Condition/Query پاس داده می‌شود (self، target، enemy، نقشه و callback‌های انتخاب بازیکن).	gameData.h
پیاده‌سازی اثرات کارت‌ها: آسیب (dmgEffect)، دفاع (defEffect)، حرکت (moveEffect)، کشیدن کارت (drawEffect)، حذف کارت، احیای کمک‌کار و غیره.	effects/*.h
شرط‌هایی مثل نزدیکی به دشمن، نزدیکی به قهرمان، باخت یا برد آخرین نبرد.	conditions/*.h
مقادیر محاسبه‌شده مانند شمارش جنگجویان یا Boost یک کارت.	queries/*.h

src/factory و include/factory ۷.۴

همه‌ی Factory‌ها ساختار یکسانی دارند: ورودی خام (یک JSON یا مسیر فایل) را می‌گیرند، یک شیء Model/Engine می‌سازند و بدنه را در try/catch قرار می‌دهند تا در صورت خطا، یک FactoryException با اطلاعات کامل پرتاب کنند.

توضیح	فایل
ساخت Hero از data/<نام>/stats.json به همراه کمک‌کارها و دسته‌ی کارت‌ها.	heroFactory
ساخت Deck با بارگذاری هر فایل JSON کارت در پوشه‌ی deck / یک قهرمان و بُرزدن آن‌ها.	deckFactory
ساخت یک کارت و نگاشت رشته‌های JSON (نوع اثر، شرط، کوئری) به کلاس‌های ++C متناظر.	cardFactory / effectFactory / conditionFactory / queryFactory
ساخت Map از data/maps/<نام>.json و ساخت هر Tile با اعتبارسنجی فیلدهای لازم.	mapFactory / tileFactory
ساخت InfoHero برای صفحه‌ی انتخاب قهرمان از فایل view.json.	heroInfoFactory

src/view و include/view ۷.۵

توضیح	فایل
کلاس پایه‌ی انتزاعی هر صحنه: onEnter/onExit/Update/Draw، به همراه	Scene.h

توضیح	فایل
دسترسی مشترک به مدیرهای صوت/فونت/بافت.	
منوی اصلی: عنوان، نقل قول ها و دکمه های شروع بازی، مشاهده ی مجموعه، تنظیمات و خروج.	MainScene
انتخاب قهرمان برای هر بازیکن و پخش صداهای مرتبط.	HeroSelectionScene
صفحه ی اصلی بازی؛ ساخت GameManager واقعی، رسم نقشه/دست کارت ها/دکمه ها و گوش دادن به رویدادها به عنوان IGameObserver.	GameScene
ساختارهای ساده ی رابط کاربری برای دکمه های قابل کلیک و ذرات پس زمینه.	ui/Button.h, ui/Particle.h

۷.۶ src/utility و include/utility

توضیح	فایل
سلسله مراتب استثنای اختصاصی پروژه به همراه توابع کمکی json_util برای اعتبارسنجی فیلدهای JSON.	exceptions.h
دو تابع پایه: load(path) برای باز و پارس کردن یک فایل JSON، و listFiles(folder) برای لیست کردن فایل های یک پوشه.	file.h/.cpp

include/libraries ۷.۷

کتابخانه های تک فایل ی شخص ثالث که مستقیماً در پروژه قرار داده شده اند (نه از طریق CMake دریافت می شوند): magic_enum.hpp برای تبدیل enum به رشته و برعکس در زمان کامپایل، و IconsFontAwesome6.h برای کدهای آیکون 6 Font Awesome در فونت رابط کاربری.

۷.۸ /data

داده های بازی به طور کامل JSON هستند و هیچ محتوایی در ++C هاردکد نشده است:

data/	
└─ Dracula/	
├─ stats.json	# لیست کمک کارها
├─ view.json	# اطلاعات نمایشی صفحه ی انتخاب قهرمان
├─ sidekicks/*.json	# آمار هر کمک کار
├─ deck/*.json	# یک فایل به ازای هر کارت یکتا
├─ SherlockHolms/	# همان ساختار
├─ maps/	
├─ baskervilleManor.json	
└─ sarpedonMap.json	# هر کدام: نام + لیست خانه ها

۸. جمع‌بندی و نتیجه‌گیری

در این پروژه، یک موتور بازی کارتی برای Unmatched با تمرکز بر جداسازی مسئولیت‌ها میان لایه‌های Model، Engine، Factory، Controller و View پیاده‌سازی شد. رویکرد داده‌محور بودن محتوای بازی (خواندن قهرمان‌ها، کارت‌ها و نقشه‌ها از JSON) باعث شد افزودن محتوای جدید بدون تغییر در کد ++C ممکن باشد. همچنین سلسله‌مراتب استثنای اختصاصی، مدیریت خطا را در تمام لایه‌ها به شکلی یکپارچه و قابل‌ردیابی درآورد.

استفاده از الگوهای طراحی استاندارد مانند Factory، Observer و چندریختی به سبک Strategy، پروژه را برای توسعه‌های آینده (مانند افزودن قهرمان‌های جدید یا اثرات کارتی پیچیده‌تر) آماده نگه داشته است.

محدودیت‌های شناخته‌شده

- برخی صحنه‌ها مانند Settings، HowToPlay و Collection هنوز پیاده‌سازی نشده‌اند و فقط در enum مربوط به نوع صحنه (ScenesType) رزرو شده‌اند.
- پروژه در حال حاضر فقط دو قهرمان (Dracula و Sherlock Holmes) و دو نقشه دارد؛ افزودن قهرمان‌های بیشتر نیازمند فایل‌های JSON مشابه است، نه تغییر کد.